

AssignmentNumber:8.4 (Present assignment number)/24(Total number of assignments)		
Q.No.	Question	Expected Time to complete
1	Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases Lab Objectives: - To introduce students to test-driven development (TDD) using AI code generation tools. - To enable the generation of test cases before writing code implementations. - To reinforce the importance of testing, validation, and error handling. - To encourage writing clean and reliable code based on AI-generated test expectations. Lab Outcomes (LOs): By the end of this lab, students will be able to: - Apply TDD methodology using AI tools. - Generate test cases before writing the actual code logic. - Validate and refactor code based on test outcomes. - Use Python's unittest or pytest libraries for test-driven development. - Develop confidence in debugging and improving code with AI guidance.	Week 4
	Task 1: Developing a Utility Function Using TDD Scenario You are working on a small utility library for a larger software system. One of the required functions should calculate the square of a given number, and correctness is critical because other modules depend on it. Task Description Following the Test Driven Development (TDD) approach: - First, write unit test cases to verify that a function correctly returns the square of a number for multiple inputs. - After defining the test cases, use GitHub Copilot or Cursor AI to generate the function implementation so that all tests pass. Ensure that the function is written only after the tests are created.	

Expected Outcome

- A separate test file and implementation file
- Clearly written test cases executed before implementation
- AI-assisted function implementation that passes all tests
- Demonstration of the TDD cycle: *test* → *fail* → *implement* → *pass*

Prompt:-

First, create a file test_square.py and write unit tests for a function square(num) using Python unittest.

Test positive, negative, zero, and decimal numbers.

Do not write the function yet.

After the tests are written, create square.py and implement square(num) so that all tests pass.

Code:-

```
lab 8.4 > test_square.py > TestSquare > test_negative_number
1  import unittest
2  from square import square
3
4  class TestSquare(unittest.TestCase):
5
6      def test_positive_number(self):
7          self.assertEqual(square(5), 25)
8          self.assertEqual(square(3), 9)
9
10     def test_negative_number(self):
11         self.assertEqual(square(-5), 25)
12         self.assertEqual(square(-3), 9)
13
14     def test_zero(self):
15         self.assertEqual(square(0), 0)
16
17     def test_decimal_number(self):
18         self.assertEqual(square(2.5), 6.25)
19         self.assertEqual(square(1.5), 2.25)
20
21
22  if __name__ == '__main__':
23      unittest.main()
```

....

Ran 4 tests in 0.001s

Ran 4 tests in 0.001s

OK

Explanation:-

The four dots indicate that four test cases were executed and passed. Ran 4 tests means the program tested the function four times with different inputs. ok means there are no errors and the square() function is working correctly.

Task 2: Email Validation for a User Registration System

Scenario

You are developing the backend of a user registration system. One requirement is to validate user email addresses before storing them in the database.

Task Description

Apply **Test Driven Development** by:

1. Writing unit test cases that define valid and invalid email formats (e.g., missing @, missing domain, incorrect structure).
2. Using **AI assistance** to implement the validate_email() function based strictly on the behavior described by the test cases.

The implementation should be driven entirely by the test expectations.

Expected Outcome

- Well-defined unit tests using unittest or pytest
- An AI-generated email validation function
- All test cases passing successfully
- Clear alignment between test cases and function behavior

Prompt:-

Follow Test Driven Development (TDD).

First, create a file test_email.py and write unit tests using unittest for a function validate_email(email).

Test valid and invalid emails (missing @, missing domain, missing username, wrong format, spaces, etc.).

Do not write the function yet.

After writing the tests, create email_validator.py and implement validate_email() so that all tests pass.

Follow the cycle: test → fail → implement → pass.

lab 8.4 > test_email.py > TestEmailValidation

```
1 import unittest
2 from email_validator import validate_email
3 class TestEmailValidation(unittest.TestCase):
4     def test_valid_emails(self):
5         self.assertTrue(validate_email("user@example.com"))
6         self.assertTrue(validate_email("john.doe@gmail.com"))
7         self.assertTrue(validate_email("abc123@yahoo.in"))
8
9     def test_missing_at_symbol(self):
10        self.assertFalse(validate_email("userexample.com"))
11
12    def test_missing_domain(self):
13        self.assertFalse(validate_email("user@"))
14
15    def test_missing_username(self):
16        self.assertFalse(validate_email("@example.com"))
17
18    def test_missing_dot_in_domain(self):
19        self.assertFalse(validate_email("user@examplecom"))
20
21    def test_space_in_email(self):
22        self.assertFalse(validate_email("user @example.com"))
23
24
25 if __name__ == "__main__":
26     unittest.main()
```

.....

Ran 6 tests in 0.001s

OK

PS C:\AIAC>

Explanation:-

We first wrote unit tests to define valid and invalid email formats. Then, based on these tests, the `validate_email()` function was implemented. After implementation, all test cases passed successfully. This shows correct use of Test Driven Development.

Task 3: Decision Logic Development Using TDD

Scenario

In a grading or evaluation module, a function is required to determine the maximum value among three inputs. Accuracy is essential, as incorrect results could affect downstream decision logic.

Task Description

Using the **TDD methodology**:

1. Write test cases that describe the expected output for different combinations of three numbers.
2. Prompt **GitHub Copilot** or **Cursor AI** to implement the function logic based on the written tests.

Avoid writing any logic before test cases are completed.

Expected Outcome

- Comprehensive test cases covering normal and edge cases
- AI-generated function implementation
- Passing test results demonstrating correctness
- Evidence that logic was derived from tests, not assumptions

Prompt:-create a file test_max_three.py and write unit tests using unittest for a function max_of_three(a, b, c). Test positive numbers, negative numbers, equal values, mixed values, and edge cases. After completing the tests, create a separate file max_three.py and implement max_of_three() so that all tests pass. Keep test code and implementation code in their respective files only.

```
lab 8.4 > test_max_three.py > ...
1  import unittest
2  from max_three import max_of_three
3  class TestMaxOfThree(unittest.TestCase):
4
5      def test_all_positive(self):
6          self.assertEqual(max_of_three(3, 7, 5), 7)
7
8      def test_all_negative(self):
9          self.assertEqual(max_of_three(-3, -7, -5), -3)
10
11     def test_mixed_values(self):
12         self.assertEqual(max_of_three(-2, 4, 1), 4)
13
14     def test_all_equal(self):
15         self.assertEqual(max_of_three(5, 5, 5), 5)
16
17     def test_two_equal_max(self):
18         self.assertEqual(max_of_three(6, 6, 2), 6)
19
20     def test_decimal_values(self):
21         self.assertEqual(max_of_three(2.5, 3.7, 3.1), 3.7)
22
23
24     if __name__ == "__main__":
25         unittest.main()
```

.....

Ran 6 tests in 0.001s

OK

Explanation:-

The function `max_of_three(a, b, c)` is used to find the maximum value among three numbers. first, it checks if `a` is greater than or equal to both `b` and `c`. If this condition is true, it returns `a` as the largest number. If not, it then checks if `b` is greater than or equal to both `a` and `c`. If this is true, it returns `b` as the largest number. If neither `a` nor `b` is the largest, then `c` must be the greatest value, so the function returns `c`.

Task 4: Shopping Cart Development with AI-Assisted TDD**Scenario**

You are building a simple shopping cart module for an e-commerce application. The cart must support adding items, removing items, and calculating the total price accurately.

Task Description

Follow a **test-driven approach**:

1. Write unit tests for each required behavior:
 - Adding an item
 - Removing an item
 - Calculating the total price
2. After defining all tests, use **AI tools** to generate the `ShoppingCart` class and its methods so that the tests pass.

Focus on behavior-driven testing rather than implementation details.

Expected Outcome

- Unit tests defining expected shopping cart behavior
- AI-generated class implementation
- All tests passing successfully
- Clear demonstration of TDD applied to a class-based design

Prompt:-First, create a file `test_cart.py` and write unit tests using `unittest` for a `ShoppingCart` class. Test adding items, removing items, and calculating total price. After writing tests, create `shopping_cart.py` and implement the `ShoppingCart` class so that all tests pass.

Code:-

lab 8.4 > test_cart.py > TestShoppingCart > test_remove_non_existing_item

```
1 import unittest
2 from shopping_cart import ShoppingCart
3 class TestShoppingCart(unittest.TestCase):
4     def setUp(self):
5         self.cart = ShoppingCart()
6
7     def test_add_item(self):
8         self.cart.add_item("Book", 100)
9         self.assertEqual(len(self.cart.items), 1)
10
11    def test_remove_item(self):
12        self.cart.add_item("Pen", 20)
13        self.cart.remove_item("Pen")
14        self.assertEqual(len(self.cart.items), 0)
15
16    def test_total_price(self):
17        self.cart.add_item("Book", 100)
18        self.cart.add_item("Pen", 20)
19        self.assertEqual(self.cart.get_total(), 120)
20
21    def test_remove_non_existing_item(self):
22        self.cart.add_item("Book", 100)
23        self.cart.remove_item("Pen")
24        self.assertEqual(self.cart.get_total(), 100)
25
26 if __name__ == "__main__":
27     unittest.main()
```

....

Ran 4 tests in 0.000s

OK

Explanation:-

The unit tests were written first to define the expected behavior of the shopping cart.

These tests checked adding items, removing items, and calculating the total price.

Initially, the tests failed because the ShoppingCart class was not implemented. After that, the class and its methods were written according to the test requirements.

When the tests were run again, all test cases passed successfully.

This shows that the implementation was developed based on test cases, following the Test Driven Development process.

Task 5: String Validation Module Using TDD

Scenario

You are working on a text-processing module where a function is required to identify whether a given string is a palindrome. The function must handle different cases and inputs reliably.

Task Description

Using **Test Driven Development**:

1. Write test cases for a palindrome checker covering:

- Simple palindromes
 - Non-palindromes
 - Case variations
2. Use **GitHub Copilot or Cursor AI** to generate the `is_palindrome()` function based on the test case expectations.

The function should be implemented only after tests are written.

Expected Outcome

- Clearly written test cases defining expected behavior
- AI-assisted implementation of the palindrome checker
- All test cases passing successfully
- Evidence of TDD methodology applied correctly

Prompt:-

First, create a file `test_palindrome.py` and write unit tests using `unittest` for a function `is_palindrome(text)`. Test simple palindromes, non-palindromes, and case variations. After writing the tests, create `palindrome.py` and implement `is_palindrome()` so that all tests pass..

Code:-

```
assign-1.4.py  assign-3.4.py  assign-5.4.py  test_squa
lab 8.4 > test_palindrome.py > ...
1  import unittest
2  from palindrome import is_palindrome
3  class TestPalindrome(unittest.TestCase):
4      def test_simple_palindrome(self):
5          self.assertTrue(is_palindrome("madam"))
6          self.assertTrue(is_palindrome("level"))
7
8      def test_not_palindrome(self):
9          self.assertFalse(is_palindrome("hello"))
10         self.assertFalse(is_palindrome("python"))
11
12         def test_case_variation(self):
13             self.assertTrue(is_palindrome("Madam"))
14             self.assertTrue(is_palindrome("RaceCar"))
15
16         def test_single_character(self):
17             self.assertTrue(is_palindrome("a"))
18
19         def test_empty_string(self):
20             self.assertTrue(is_palindrome(""))
21
22     if __name__ == "__main__":
23         unittest.main()
```


.....

Ran 5 tests in 0.000s

OK

----- □
Explanation:-

First, unit tests were written in test_palindrome.py to define the expected behavior of the palindrome checker. These tests verify simple palindromes, non-palindromes, and case variations. Initially, the tests failed because the is_palindrome() function was not implemented. After that, the function was written in palindrome.py according to the test requirements. The function converts the input string to lowercase and compares it with its reverse. If both are equal, it returns True; otherwise, it returns False. After implementation, all test cases passed successfully.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots